AMRITA VISHWA VIDYAPEETHAM AMRITAPURI CAMPUS

Course Project Proposal

S5 EAC (Academic Year 2025–2026)

23ECE313: Embedded Systems

1. Project Title

AutoFuzzer: An Embedded Communication Fuzzer for Automated Firmware Robustness Testing

2.Student Details

Student Names: Abishek R Nair, Alwin Varghese

Roll Numbers: AM.EN.U4EAC24004, AM.EN.U4EAC24009

Course: Embedded Systems

Semester: 5

Faculty Guide: Dr Rajesh Kannan

3. Abstract

Embedded systems use communication interfaces such as SPI, UART, I²C, and CAN to allow data exchange between microcontrollers, sensors and peripheral devices. Although functional testing verifies correct operation under normal conditions, it often fails to assess how the firmware will behave when it is exposed to the malformed, boundary-case or unexpected communication. This project presents 'AutoFuzzer' an embedded communication fuzzer system based on ESP32, which is a low-cost and capable of automatically generating and transmitting malformed, boundary-value, mutated and high-throughput packets to evaluate and check the robustness of communication-handling firmware. The system uses a single ESP32 DevKit V1 as the fuzzing controller and health monitor, while an STM32 Blue Pill or Nucleo board will serves as the device under test, running its normal application firmware.

AutoFuzzer has mainly four complementary fuzzing modes: random, boundary, protocol mutation, and high speed stress testing. Protocol mutation systematically changes valid packets by altering command bytes, payload, packet length, checksums, and address fields. The STM32 will continuously send a heartbeat signal through GPIO / UART to detect firmware crashes or lockups. The ESP32 will be monitoring the signal in real time. Failure is defined as the loss of heartbeat within a pre-defined time out. The triggering packet will be logged automatically for analysis. Test results including protocol, packet number, mutation type and pass/fail status are displayed on the serial monitor and can optionally be visualised on an OLED display with LED and buzzer alerts. AutoFuzzer is a low-cost, reusable platform for firmware validation, embedded systems research, and communication robustness testing. The modular architecture allows for the inclusion of new communication protocols and mutation strategies.

4. Introduction

Embedded systems forms the computational backbone of modern electronics, spanning everything from industrial controllers to consumer IoT devices, and they rely on low-level communication protocols such as SPI, UART, I²C, and CAN to move data between microcontrollers, sensors, and peripherals. Because these protocols operate close to the hardware and lack the error-handling sophistication built into higher-layer network stacks, firmware has to be exceptionally robust: a single malformed byte or an unexpected timing pattern is enough to trigger a crash, a hang, or silent data corruption in a device that may be operating in a safety-critical or physically inaccessible environment.

Firmware robustness and communication reliability are therefore central design requirements rather than afterthoughts. Unlike desktop software, embedded firmware often runs without memory protection, watchdog infrastructure, or crash-reporting facilities, which means failures can go undetected until real damage has already occurred. Fuzz testing has emerged as an effective way to expose such weaknesses, automatically generating malformed, boundary-value, mutated, or high-throughput inputs and observing how the system responds, in the process revealing edge cases that manual test design rarely anticipates.

Conventional firmware testing, however, still leans heavily on manual test-case authoring and ad hoc debugging, an approach that is labor-intensive and scales poorly once multiple communication interfaces or firmware revisions enter the picture. It is this gap, between the need for systematic robustness testing and the limits of manual approaches, that motivates AutoFuzzer: an ESP32-controlled, STM32-tested platform implementing random, boundary-value, mutation, and stress-testing modes across SPI, UART, I²C, and CAN, with heartbeat-based crash detection for automated firmware validation.

5. Literature Review

Protocol fuzzing has matured into a distinct research area in its own right, with systematic surveys now bringing together hundreds of publications and pointing to challenges that go beyond general software fuzzing, particularly those tied to stateful protocol behavior [1]. Surveys on bare-metal firmware fuzzing raise a parallel set of concerns, noting that hardware complexity, platform diversity, and asynchronous peripheral behavior all complicate robustness testing, with most existing research concentrated on emulation and rehosting rather than on the input-generation, mutation, and scheduling strategies more typical of generic software fuzzing [2]. This body of work sits within a longer tradition of embedded communication and hardware design, including secure real-time channel communication engines [3] and edge-enabled heterogeneous sensor networks [4], both of which point to the continuing importance of reliable low-level communication in embedded platforms.

A substantial cluster of research takes up firmware rehosting and emulation as a route to fuzzing entire firmware images. Since embedded network interfaces represent a major attack surface, rehosting emulates the underlying physical hardware to enable full-firmware fuzzing, though complex, multi-layered protocol formats still tend to limit how deeply such tools can surface nested bugs [5]. Related work narrows the peripheral-modeling problem through adaptive combinations of fuzzing and symbolic execution [6], and through firmware-guided

interrupt modeling that cuts down on the need for manual intervention [7]. Other approaches sidestep software emulation altogether: EmbedFuzz transplants binary firmware onto compatible hardware for native-speed execution [8], SHiFT fuzzes firmware natively on the microcontroller itself for higher fidelity and fewer false positives [9], and TaPaFuzz accelerates RISC-V fuzzing through FPGA-based in-hardware monitoring, reaching speedups of roughly 19× over emulation [10]. These efforts run alongside custom controller hardware work such as PCB design for motor control and robotic master controllers [11], [12].

Stateful, system-level testing is also gaining ground as a priority. STAFF targets inter-process dependencies that conventional single-process fuzzers tend to overlook [13], while reviews of hardware fuzzing frame the technique as a scalable alternative to costly formal verification, alongside adjacent embedded-security work on side-channel attack detection [15] and low-power asynchronous architectures [16] [14]. Taken together, this literature places its emphasis squarely on emulation fidelity and system-level complexity [1], [2], [5]–[10], [13], [14], leaving comparatively little attention on lightweight, board-to-board protocol fuzzing — the gap that AutoFuzzer sets out to address.

6.References

[1] X. Zhang, C. Zhang, X. Li, Z. Du, B. Mao, Y. Li, Y. Zheng, Y. Li, L. Pan, Y. Liu, and R. H. Deng, "A Survey of Protocol Fuzzing," *ACM Computing Surveys*, vol. 57, no. 2, pp. 1–36, 2024, doi: 10.1145/3696788.

[2] Asmita, R. Tsang, S. Ghimire, S. Salehi, and H. Homayoun, "Bare-Metal Firmware Fuzzing: A Survey of Techniques and Approaches," *IEEE Access*, vol. 13, pp. 98253–98277, 2025, doi: 10.1109/ACCESS.2025.3575691.

[3] R. K. Megalingam, "A Dynamic Real Time Crypto Engine Implementation for Secure Channel Communication," in *Proc. ICSIM Conf.*, Indian Institute of Management Ahmedabad, India, 2009.

[4] K. S. Kumar, G. Dheeraj, and R. K. Megalingam, "Design and Evaluation of an Edge-Enabled Wireless Sensor Network Using Heterogeneous Multi-Core Architecture on Raspberry Pi," in *Proc. 2026 7th Int. Conf. Mobile Computing and Sustainable Informatics (ICMCSI)*, Goathgaun, Nepal, 2026, pp. 620–625, doi: 10.1109/ICMCSI67283.2026.11412937.

[5] M. Bley, T. Scharnowski, S. Wörner, M. Schloegel, and T. Holz, "Protocol-Aware Firmware Rehosting for Effective Fuzzing of Embedded Network Stacks," in *Proc. 2025 ACM SIGSAC Conf. Computer and Communications Security (CCS '25)*, 2025, pp. 4484–4498, doi: 10.1145/3719027.3765125.

[6] J. Kim, J. Yu, Y. Lee, D. D. Kim, and J. Yun, "HD-FUZZ: Hardware Dependency-Aware Firmware Fuzzing via Hybrid MMIO Modeling," *Journal of Network and Computer Applications*, vol. 224, Art. 103835, 2024, doi: 10.1016/j.jnca.2024.103835.

[7] Y. Wei, Y. Wang, L. Zhou, X. Zhou, and Z. Jiang, "IEmu: Interrupt Modeling from the Logic Hidden in the Firmware," *Journal of Systems Architecture*, vol. 154, Art. 103237, 2024, doi: 10.1016/j.sysarc.2024.103237.

[8] F. Hofhammer, Q. Wang, A. Bhattacharyya, M. Salehi, B. Crispo, M. Egele, M. Payer, and M. Busch, "EmbedFuzz: High Speed Fuzzing Through Transplantation," arXiv:2412.12746, 2024.

[9] A. Mera, C. Liu, R. Sun, E. Kirda, and L. Lu, "SHiFT: Semi-hosted Fuzz Testing for Embedded Applications," in *Proc. 33rd USENIX Security Symp.*, Philadelphia, PA, USA, 2024, pp. 5323–5340.

[10] F. Meisel, C. Spang, D. Volz, and A. Koch, "TaPaFuzz: Hardware-Accelerated RISC-V Bare-Metal Firmware Fuzzing Using Rapid Job Launches," *Journal of Systems Architecture*, vol. 156, Art. 103288, 2024, doi: 10.1016/j.sysarc.2024.103288.

[11] R. K. Megalingam, S. K. Manoharan, S. M. Mohandas, and A. T. Kamalasanan, "Design and Development of a Controller PCB for a Dual BLDC-Based Differential Drive Rover," in *Algorithms for Intelligent Systems*, Singapore: Springer Nature Singapore, 2025, doi: 10.1007/978-981-96-0228-5_10.

[12] S. Vadivel, S. Vinayagam, A. Sreelatha, and R. K. Megalingam, "Design and Development of Master Controller for Robotic Applications," in *Proc. IEEE ASIANCON*, 2024, doi: 10.1109/ASIANCON62057.2024.10837729.

[13] A. Izzillo, R. Lazzeretti, and E. Coppa, "STAFF: Stateful Taint-Assisted Full-system Firmware Fuzzing," *Computers & Security*, vol. 170, Art. 105003, 2026, doi: 10.1016/j.cose.2026.105003.

[14] R. Saravanan and S. M. Pudukotai Dinakarrao, "The Emergence of Hardware Fuzzing: A Critical Review of Its Significance," arXiv:2403.12812, 2024.

[15] N. Radhakrishnan, A. Pradeepkumar, and R. K. Megalingam, "ML-Based Detector for Microarchitectural and Side-Channel Attacks on Embedded IoT Devices," in *Proc. 2026 Int. Conf. Mobile Computing and Sustainable Informatics (ICMCSI)*, 2026, doi: 10.1109/ICMCSI67283.2026.11412606.

[16] Amrutha, K. T. R. Shriya, and R. K. Megalingam, "Event-Driven IoT Node: A Low-Power Asynchronous Microcontroller Architecture," in *Proc. 2026 IEEE Int. Conf. Interdisciplinary Approaches in Technology and Management for Social Innovation (IATMSI)*, IEEE, 2026, doi: 10.1109/IATMSI68868.2026.11466136.